INDIRA UNIVERSITY, PUNE

School of Information Technology

B.Sc. (Artificial Intelligence & Machine Learning) — Semester III

MINI PROJECT SYNOPSIS

(Generative AI-based Tool)

1. Project Information

Project Title	C.I.D – Code Intelligent Debugger
Team Members (Name & Roll No.)	1. Anuj Gore - 14 2. Shubh Srivastav – 75 3. Danish Khan - 25
Class / Division	S.Y. B.Sc. (AI & ML)
Guide / Mentor Name	Sarita Byagar
Academic Year	2026–27
Date of Submission	06/08/2026

2. Introduction

C.I.D (Code Intelligent Debugger) is a web-based AI platform that leverages Large Language Models (LLMs) to help developers understand, debug, optimise, and secure their source code. The system accepts code in 15+ programming languages and returns structured, human-readable analysis results within seconds.

Traditional static analysis tools rely on predefined rule sets and fail to understand the semantic intent of code. LLMs, trained on vast repositories of code and documentation, can reason contextually — identifying logic errors, explaining complex algorithms, and detecting security vulnerabilities that rule-based tools miss entirely.

C.I.D is built using a Python Flask backend, Monaco Editor frontend, and the Groq inference API running LLaMA 3.3 70B — delivering professional-grade code analysis at zero cost on the free tier. The project demonstrates practical application of LLM integration, REST API design, and full-stack web development.

3. Problem Statement

- Developers face persistent challenges in code comprehension, debugging, and quality assurance that existing tools do not adequately address.
- Developers spend approximately 58% of their working time reading and understanding existing code rather than writing new functionality. This is especially problematic when dealing with legacy codebases, unfamiliar languages, or undocumented projects where context is absent.
- Junior developers and students lack access to real-time mentorship for debugging. When they encounter errors, the feedback loop is slow — waiting for a senior developer or instructor to be available creates delays and frustration that slows the learning process significantly.

- Rule-based linters such as ESLint and Pylint detect syntax errors but cannot identify logic bugs or understand the semantic intent of code. A function that runs without errors but produces incorrect output is invisible to these tools, yet this is the most common type of bug in real-world software.
- No single free tool currently combines code explanation, bug detection, complexity analysis, and security scanning in one unified interface. Developers must switch between multiple specialised tools, increasing cognitive load and fragmenting their workflow.
- Security vulnerabilities such as SQL injection, cross-site scripting, and hardcoded credentials frequently go undetected during manual code reviews, particularly in small teams without dedicated security engineers.

4. Objectives

- Build a web-based code analysis platform powered by a state-of-the-art LLM.
- Provide line-by-line explanation of code in plain, beginner-friendly English.
- Automatically detect bugs (syntax, logic, runtime) and suggest corrected code.
- Analyse time and space complexity using Big O notation with optimisation tips.
- Identify security vulnerabilities aligned with the OWASP Top 10 framework.
- Support 15+ programming languages with automatic language detection.
- Persist analysis history in SQLite for user reference and statistics.
- Deploy the application publicly on free-tier cloud hosting (Render / Railway).

5. Scope of the Project

The project covers a web-based code analysis tool supporting four distinct analysis modes — Explain, Debug, Optimise, and Security Scan — for a minimum of fifteen programming languages including Python, JavaScript, Java, C++, Go, Rust, PHP, and others. The system accepts code snippets of up to 50,000 characters, processes them through a carefully engineered LLM prompt pipeline, and returns structured JSON results rendered in a professional split-screen interface.

The application includes a Monaco Editor for code input (the same editor used in VS Code), automatic programming language detection using pattern matching, SQLite-backed session history with pagination and filtering, and a dark/light theme toggle with glassmorphism design.

It does not cover live code execution in a sandboxed environment for languages other than Python and JavaScript, IDE plugin development, compiler-level or bytecode analysis, real-time collaborative multi-user editing, native mobile applications, or user authentication with multi-tenant account management. These features are planned for version 2.0.

Target users are students learning to code, junior developers seeking debugging assistance, educators teaching programming concepts, and developers performing code reviews.

6. Literature Survey / Existing Systems

Sr. No.	Existing Tool / Paper	Key Feature	Limitation / Gap
1	GitHub Copilot (GitHub & OpenAI, 2021)	AI-powered code generation and autocomplete integrated into IDEs	Focuses only on code generation; does not provide explanation, debugging, or security analysis; requires paid subscription
2	SonarQube (SonarSource)	Rule-based static code analysis for bugs, code smells, and vulnerabilities	Cannot understand semantic intent of code; misses logic errors; requires server setup and configuration
3	ChatGPT (OpenAI, 2022)	General-purpose conversational AI capable of code explanation	Not purpose-built for code analysis; no structured output format; no integrated code editor; requires manual copy-paste workflow
4	Research: "Evaluating Large Language Models Trained on Code" (Chen et al., 2021)	Introduced Codex model fine-tuned on 159GB of Python code; established HumanEval benchmark	Research-only; demonstrated feasibility but not deployed as an accessible code analysis tool
5	Amazon CodeWhisperer (AWS, 2022)	Code generation with integrated security scanning using AWS threat intelligence	Tightly coupled to AWS ecosystem; security scanning limited to generation-time; not a standalone analysis platform

[C.I.D addresses the gaps identified above by combining explanation, debugging, optimization, and security analysis into a single free web-based interface powered by LLM reasoning rather than rule-based pattern matching.]

7. Proposed System / Methodology

- C.I.D operates as a real-time code analysis pipeline layered on top of an LLM-based reasoning engine. When a user submits code through the Monaco Editor interface, the system follows a structured processing workflow.
- The user pastes source code and selects the desired analysis mode (Explain, Debug, Optimise, or Security). The JavaScript frontend sends an HTTP POST request to the Flask API with the code, selected language, and mode as a JSON payload.
- The Flask backend validates the input through the validator's module — checking for empty submissions, excessive length, unsupported languages, and invalid mode values. If the language is set to Auto Detect, a regex-based language detection module analyses the code against language-specific patterns for fifteen languages and assigns the most probable match with a confidence score.
- The prompt builder module then constructs a carefully engineered prompt that instructs the LLM to act as a specialised code analyst and respond in a strictly defined JSON schema. Different prompt templates are used for each mode — the explain prompt requests line-by-line breakdown, the debug prompt requests bug identification with severity ratings, the optimise prompt requests Big O analysis with improved code, and the security prompt requests OWASP-aligned vulnerability detection.
- The constructed prompt is sent to the Groq API which runs the LLaMA 3.3 70B model. Groq's custom LPU (Language Processing Unit) hardware typically returns responses in 1 to 3 seconds.

- The response parser module extracts valid JSON from the LLM output using a multi-stage parsing pipeline — direct JSON parsing, markdown code block extraction, regex-based JSON location, and common formatting issue correction. If all parsing stages fail, a structured fallback response is returned to ensure the frontend always receives a consistent data shape.
- The parsed result is persisted to the SQLite database as a CodeSession record and returned to the frontend, where the JavaScript renderer constructs mode-appropriate HTML — numbered line explanations for Explain mode, severity-coded bug cards for Debug mode, complexity comparison grids for Optimise mode, and OWASP-tagged vulnerability reports for Security mode.

8. Proposed Tools & Technologies

Category	Tool / Technology Proposed
Programming Language	Python 3.14
Generative AI Model / API	Groq API with LLaMA 3.3 70B Versatile
Web Framework	Flask 3.0.3
Frontend	HTML5, CSS3, JavaScript, Monaco Editor
Database / ORM	SQLite with SQLAlchemy
Rate Limiting	Flask-Limiter
Input Sanitisation	Bleach
Environment Management	python-dotenv, venv
HTTP Client	requests, httpx
Syntax Highlighting	Pygments
Version Control	Git & GitHub
IDE	Visual Studio Code
Deployment Platform	Render / Railway (free tier)

9. System Requirements

Hardware Requirements	Software Requirements
Laptop/PC — 8GB RAM minimum, Intel Core i5 or above, 5GB free disk space, broadband internet connection	Python 3.10+, VS Code, modern web browser (Chrome/Firefox/Edge), Groq API key (free), Git

10. Expected Outcomes

- A fully functional web application deployed on a public cloud URL capable of handling real-time code analysis requests. The system is expected to support 15+ programming languages with automatic language detection achieving greater than 90% accuracy on standard code patterns.

- Average explanation quality is targeted above 85% accuracy based on structured test cases comparing LLM output against expert-authored explanations. The system should deliver responses within 3 seconds under normal network conditions using the Groq inference API.

- The project will demonstrate a well-documented, modular codebase following industry-standard software engineering practices including separation of concerns, service-layer architecture, input validation, rate limiting, and comprehensive error handling.

- A reusable academic deliverable demonstrating practical application of LLM technology, REST API design, prompt engineering, and full-stack web development.

11. Applications / Real-World Relevance

This tool is directly applicable to educational institutions teaching programming where students can receive instant AI explanations of assignment code without waiting for instructor availability. It can be adapted for coding bootcamps providing on-demand debugging assistance to learners, for software companies accelerating developer onboarding by generating explanations of existing codebases, and for open source project maintainers reviewing contributed code for correctness and security vulnerabilities.

It is also relevant for interview preparation where candidates can analyse their algorithmic solutions receiving complexity and correctness feedback instantly, for legacy code documentation where organisations generate automated explanations for undocumented systems, and for freelance developers who need to explain implemented functionality to non-technical clients.

12. Ethical Considerations & Limitations

- Code submitted to C.I.D is transmitted to the Groq API for inference. Users should be informed that submitted code is processed by third-party infrastructure and should avoid submitting proprietary, confidential, or personally identifiable code without appropriate authorisation.

- LLM-generated explanations and bug fixes must be treated as advisory rather than authoritative. The system may produce plausible but incorrect analyses, particularly for highly domain-specific or unconventional code patterns. Human verification remains essential before acting on AI suggestions in production environments.

- Known limitations include Groq free tier rate limits of 30 requests per minute and 14,400 requests per day, the inability to execute code for runtime verification of generated fixes (except Python and JavaScript), reduced language detection accuracy for code snippets under 10 lines, potential LLM bias toward English-language coding conventions, and non-deterministic response variation between requests for identical input. The security scanner identifies common vulnerability patterns but cannot guarantee comprehensive coverage of all possible attack vectors — it is therefore restricted to advisory scanning only, with all final security decisions left to human developers.

13. References

- Chen, M., Tworek, J., Jun, H., et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021. Available: https://arxiv.org/abs/2107.03374

- Vaswani, A., Shazeer, N., Parmar, N., et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, vol. 30, 2017. Available: https://arxiv.org/abs/1706.03762

- Groq Inc., "Groq API Documentation and Model Reference," 2024. Available: https://console.groq.com/docs

- OWASP Foundation, "OWASP Top Ten 2021," 2021. Available: https://owasp.org/www-project-top-ten/

- Pallets Projects, "Flask Documentation (3.0.x)," 2024. Available: https://flask.palletsprojects.com/en/3.0.x/

- Microsoft Corporation, "Monaco Editor Documentation," 2024. Available: https://microsoft.github.io/monaco-editor/

- Meta AI, "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023. Available: https://arxiv.org/abs/2302.13971

- SQLAlchemy, "SQLAlchemy 2.0 Documentation," 2024. Available: https://docs.sqlalchemy.org/en/20/

Signature of Student